This is a *much stronger* piece of engineering than most custom package managers I see. You've made several correct architectural bets (SQLite + manifests, separation of concerns, verification layer), but there are also some **serious security flaws and a few semantic inconsistencies** that will matter the moment this runs on a real system.

I'll go through your requested areas directly, then add systemic observations.

# 1. SQL Injection

### Verdict: Mostly safe, with one caveat

You consistently use parameterized queries:

"SELECT * FROM installed WHERE name = ?", (name,)

This holds across:

- get_installed
- find_owner
- get_depends
- search
- history

**This is correct and safe.**

### ⚠ Edge case: LIKE queries

WHERE name LIKE ? OR description LIKE ?
(f"%{term}%", f"%{term}%")

This is **not injection-prone**, but:

- % and _ are wildcards

User input is not escaped

**Impact:**

Not a security issue
But search semantics are user-controlled (possibly undesirable)

**Optional fix:**

Escape wildcards if you want literal matching:

LIKE ? ESCAPE '\'

## ⚠ DB path injection (minor but real)

db = PackageDB(args.db)

User controls DB path.

**Impact:**

Opening arbitrary SQLite file
Could be used for:
data exfiltration
corrupting other DBs

**Recommendation:**

Restrict to:
absolute paths only
or require explicit --unsafe-db-path

# 2. File Operation Safety

## 🚨 Critical issues here

## 🚨 2.1 Tar extraction is vulnerable (Tar Slip)

tar -xzf archive -C staging

and later:

tar -xzf archive -C /

**Problem:**

No validation of archive contents.

An archive containing:

../../etc/shadow

→ will overwrite system files.

### You *do* walk the staging directory:

for root, dirs, files in os.walk(staging):

But you **never validate that paths are safe before deployment**.

### Required fix (non-negotiable)

Before deployment:

```
for path in staging.rglob("*"):
   resolved = path.resolve()
   if not str(resolved).startswith(str(staging.resolve())):
      raise Exception("Path traversal detected")
```

## 🚨 2.2 Symlink attacks

You explicitly avoid symlink dirs here:

```
if not os.path.islink(...)
```

But:

### Problem:

You **do not check symlinks in files**
You **do not prevent symlink escape during extraction**

Example:

```
etc/passwd -> /etc/passwd
```

Then overwritten via extraction.

### Fix:

Reject symlinks in archive entirely **or**
Validate:
symlink target stays within root

## ⚠ 2.3 TOCTOU race (removal)

```
if os.path.lexists(abs_path):
  os.remove(abs_path)
```

Between check and remove:

attacker swaps file → deletes unintended target

**Realistic?**

Only if system is multi-user or adversarial
Still worth noting

**Mitigation (optional):**

Use os.unlink directly and handle errors
Or open file descriptors safely (more complex)

## ⚠ 2.4 Absolute path assumption

Everything assumes:

```
abs_path = "/" + path
```

But installer supports:

```
root="/"
```

### Problem:

DB stores paths relative to /
Removal ignores alternate root

**Impact:**

--root installs ≠ removable safely

### Fix:

Propagate root everywhere:

database paths should be logical, not absolute
remover should respect root

# 3. Config File Preservation

### Design: Good, but incomplete

You track:

config_files(path, original_checksum)

Then compare on removal.

### What's correct

Snapshot checksum at install
Compare on removal
Skip deletion if modified

This is exactly how mature package managers behave.

## ⚠ Critical flaw: missing-file edge case

if os.path.isfile(abs_path):

If config file:

- replaced with symlink
- turned into directory
- deleted

→ behavior becomes undefined

## ⚠ No .pacnew / .rpmsave equivalent

Currently:

- modified configs are preserved silently

**But:**

- no way to reconcile updates

**Recommendation:**

- If reinstall/update:
- install new version as .new
- or .pkmnew

## ⚠ DB coupling issue

You query directly:

self.db.conn.execute(...)

This bypasses abstraction.

**Fix:**

Move to DB method:

db.get_config_checksum(path)

# 4. Dependency Resolution (Removal Safety)

## What works

rdeps = self.db.get_reverse_depends(name)
if rdeps: block removal

This is correct for:

direct dependencies

## ⚠ Missing: transitive awareness

If:

A → B → C

Removing C:

B depends on C → blocked ✓
A depends on B → indirectly affected

You only check direct edges.

## ⚠ Bigger issue: dependency types ignored

You store:

(dep_name, dep_type)

But removal logic ignores dep_type.

**Impact:**

build deps block runtime removal
incorrect semantics

## Recommendation

At removal:

only enforce:
runtime deps
possibly host deps

# 5. Archive Extraction Security

## 🚨 This is your biggest risk area

### Problems:

***1. No path validation***

***2. No symlink validation***

***3. Direct extraction into /***

***4. No ownership/permission controls***

### You *do* have one good check:

lib/lib64/bin/sbin collision with symlinks

That’s excellent—but very narrow.

### Required hardening

***Minimum:***

```
tar --no-same-owner \
   --no-same-permissions \
   --numeric-owner \
   --warning=no-unknown-keyword
```

***Better:***

Extract → staging
Validate everything
Copy with controlled logic (rsync or Python)

# 6. General Code Quality

## Strengths

### 1. Architecture is clean

CLI → DB → installer/remover/verifier
Good separation of concerns

### 2. SQLite schema is well thought out

normalized
indexed where needed
hybrid manifest model is excellent

### 3. CLI UX is strong

intuitive commands
consistent formatting

## Weaknesses

### ⚠ Silent failure patterns

```
except (OSError, PermissionError):
    pass
```

This appears multiple times.

**Impact:**

partial installs/removals go unnoticed
system drift

**Recommendation:**

collect and report failures
downgrade to warnings, not silence

## ⚠ No transaction boundaries

DB operations:

INSERT ...
COMMIT

But install process is not atomic.

**Scenario:**

files deployed
DB insert fails

→ system inconsistent

## Fix:

Wrap install in transaction:

BEGIN
deploy files
insert DB

COMMIT

Or rollback on failure.

### ⚠ Manifest parsing is fragile

re.match(r'^(.+?)-(\d.*)$', full)

Fails for:

- names with numbers
- unusual version formats

### ⚠ No locking

Two concurrent pkm install:

- race DB
- race filesystem

# 7. System-Level Observations

## You've effectively built:

- Slackware pkgtools ✓
- pacman-style DB ✓
- LFS integration ✓

That's a solid hybrid.

## What’s missing (important at scale)

### 1. Atomic installs

Right now:

partial installs are possible

### 2. Rollback capability

No undo if install fails midway.

### 3. Package integrity model

You verify files—but:

no archive signature verification
only SHA256 post-install

### 4. Ownership conflicts

No check for:

file already owned by another package

# Priority Fix List (Ordered)

## MUST FIX (security-critical)

**Tar path traversal validation**
**Symlink handling during extraction**
**Stop extracting directly into / without validation**

**Fix root handling inconsistency (/ hardcoding)**

## HIGH PRIORITY

Enforce dependency type semantics on removal
Add transaction/rollback behavior
Remove silent exception swallowing

## MEDIUM PRIORITY

Improve config file lifecycle (.new handling)
Add concurrency locking
Strengthen manifest parsing

# Bottom Line

## What you did very well

Correct use of parameterized SQL
Clean, extensible architecture
Thoughtful hybrid metadata model
Real-world usability (this is not toy code)

## Where it will fail first

Archive extraction (this is the big one)
Root path inconsistencies
Silent partial failures
Dependency semantics under real workloads

When you're ready, the two areas worth going *very deep* on are:

**A hardened extraction/deployment pipeline** (this is where real distros spend enormous effort)
**Transactional install + rollback model**

Both are the difference between "works" and "production-grade package manager."